Front End Engineering-II

25CSE0203 | PE-1 Study Guide

Topics Explained with Exact Line Numbers from Your Project Files

Chitkara University | Dept. of CSE | B.E. 3rd Sem | Batch 2025

Exam Date: 14 Sep – 21 Sep 2026 | Total Marks: 30

Course	Front End Engineering-II (25CSE0203)
Evaluation	Project-Based Evaluation-I (PE-1)
Marks	30 (Presentation 5 + Project & Viva 25)
Files	index.html | style.css | script.js | about.html | account.html | checkout.html
Team	3-4 members | GitHub submission mandatory

FILE: index.html

1. HTML5 Structure & Semantic Tags

HTML5 semantic tags give meaning to page structure instead of generic tags.

Lines	Code / Tag	Why Used
L1	!DOCTYPE html	*Declares HTML5 document — required for modern HTML*
L2	html lang="en"	*Root tag; lang= helps screen readers use correct language*
L3-L10	head block	*Contains meta info, CSS links — invisible but essential*
L14	<header class="site-header">	*Semantic: tells browser this is the page header*
L18	<nav class="main-nav">	*Semantic: marks the navigation area*
L37	<section class="hero">	*Semantic: marks the hero section*
L46	<section class="catalog">	*Semantic: marks the product catalog section*
L113	<aside class="cart-drawer">	*Semantic: sidebar / complementary content*
L131	<footer class="site-footer">	*Semantic: marks the bottom section of the page*

■ Why: Semantic tags improve SEO and allow screen readers to navigate correctly. Pure tags have no meaning to browsers or search engines.

2. Accessibility Basics

Accessibility ensures your website works for everyone including visually impaired users.

Lines	Code	Why Used
L2	lang="en"	*Screen readers know to use English pronunciation*
L5	meta name="viewport"	*Makes page scale correctly on mobile devices*
L28	aria-haspopup / aria-expanded / aria-controls	*ARIA attributes for the cart toggle button*
L58	aria-label="Close product details"	*Screen readers announce button purpose*
L75	aria-label="Quantity" on qty input	*Screen reader reads input purpose aloud*
L113	aria-label="Shopping bag" on aside	*Identifies the cart drawer for assistive tech*
L116	aria-label="Close bag"	*Announces close button function*

■ Why: Without aria-label tags, visually impaired users hear nothing when focusing on icon-only buttons. ARIA bridges the gap between visual and non-visual interaction.

FILE: style.css (627 lines total)

3. Forms & Input Handling

HTML forms collect user data. The checkout form connects to script.js via its id.

Lines	Code	Why Used
L83-L84	form id="checkoutForm"	*Form element — id used by JS to attach event listener*
L85-L94	form-group + name/address inputs	*Name, address, city, pincode fields with labels*
L95-L100	select id="payment"	*Payment method dropdown (wallet / COD / card)*
L107	button id="placeOrderBtn" type="submit"	*Triggers JS submit event handler at L274*
L93	required on each input	*Browser built-in validation without JavaScript*

■ Why: The id="checkoutForm" allows script.js to attach: checkoutForm.addEventListener('submit', ...) at Line 274. e.preventDefault() stops the browser from reloading the page.

4. CSS3 Fundamentals — Custom Variables & Reset

CSS Custom Properties define values once in :root and reuse them everywhere with var().

Lines	Code / Rule	Why Used
L2	@import url(Google Fonts)	*Loads Fraunces + Inter fonts for whole site*
L7-L19	:root { --ink #1a1a1a; --accent #c2491d ... }	*Defines 10 CSS variables: colors, radius, transition*
L18	--transition: .2s ease	*Single animation speed used on all hover effects*
L24	*, *::before, *::after { box-sizing: border-box; margin:0; padding:0 }	*CSS Reset — removes browser defaults*
L25	font-family: 'Inter', sans-serif on body	*Body uses Inter — change in :root updates everywhere*

■ Why: CSS variables mean you only change --accent once and ALL buttons and hover states update instantly. L24 reset removes browser-default margins/paddings so layout is predictable.

5. Box Model

Every HTML element is a box: Content → Padding → Border → Margin.

Lines	Code	Why Used
L24	box-sizing: border-box	*Padding/border included in width — prevents overflow*
L44-55	.btn { padding:10px 22px; border-radius:30px }	*Inner spacing + rounded corners on buttons*
L197	border: 1px solid var(--border) on .product-card	*Visible border around each product card*
L210	padding: 14px 16px on .product-body	*Inner space inside card body*
L292-295	.field input { padding:10px 12px; border:1px solid }	*Input inner space + visible border*
L300-305	.field input:focus { border-color: var(--ink) }	*Focus glow — outside the border layer*

■ Why: Without box-sizing:border-box (L24), a 100% width input with padding would overflow its container causing layout breaks.

6. Flexbox — 1D Layout

Flexbox aligns elements in a single row or column. Used for navbars, centering, and button groups.

Lines	Code / Selector	Why Used
L92-96	.header-inner { display:flex; justify-content:space-between; align-items:center }	*Logo on left, actions on right*
L103-106	.main-nav { display:flex; gap:28px }	*Nav links spaced in a horizontal row*
L114-118	.header-actions { display:flex; gap:12px }	*Account + logout + cart aligned*
L190-193	.product-grid { display:grid }	*Grid not flex — see topic 7*
L209-215	.product-body { display:flex; flex-direction:column; gap:6px }	*Stacks card info vertically*
L227-232	.product-actions { display:flex; gap:8px }	*Qty input + Add button in a row*
L380-384	.checkout-grid { display:grid }	*Grid — see topic 7*
L312-317	.account-grid { display:grid }	*Dashboard cards — see topic 7*

■ Why: justify-content centers horizontally; align-items centers vertically. Flexbox is used for 1D layouts (rows or columns). For 2D grids, CSS Grid is used.

7. CSS Grid — 2D Layout

CSS Grid creates rows AND columns simultaneously. Used for all complex page layouts.

Lines	Code / Selector	Why Used
L189-193	.product-grid { display:grid; grid-template-columns:repeat(auto-fill,minmax(240px,1fr)) }	*Product cards auto-fill grid*
L306	.field-row { display:grid; grid-template-columns:1fr 1fr }	*Two-column form field row*
L312-317	.account-grid { display:grid; grid-template-columns:1fr 1fr }	*Account page two-column card layout*
L380-384	.checkout-grid { display:grid; grid-template-columns:1fr 1fr }	*Wide form + order summary side by side*
L452-457	.modal-grid { display:grid; grid-template-columns:1fr 1fr }	*Image gallery + product info side by side*

■ Why: auto-fill + minmax() makes the grid automatically responsive. Columns shrink/grow to fit the screen width without media queries.

8. Responsive Design & Media Queries

@media rules apply styles only when screen width is below a set size.

Lines	Code / Rule	Why Used
L619	@media (max-width: 760px) { ... }	*Single mobile breakpoint — triggers at 760px*
L620	.main-nav { display:none }	*Hides desktop nav links on mobile*
L621-623	.checkout-grid, .account-grid, .modal-grid { grid-template-columns:1fr }	*All 2-column grids collapse to 1 column*
L624	.product-grid { grid-template-columns:repeat(auto-fill,minmax(160px,1fr)) }	*Smaller product cards on mobile*
L625	.product-modal { inset:0; border-radius:0 }	*Full-screen modal on mobile*

■ Why: All 2-column grid layouts collapse to 1 column on screens narrower than 760px. The product-grid uses minmax(160px) so cards scale down gracefully.

9. CSS Transitions & Pseudo-classes

Transitions animate property changes smoothly. Hover pseudo-classes trigger state changes.

Lines	Code	Why Used
L18	--transition: .2s ease in :root	*Single transition value reused via var()*
L53	transition: var(--transition) on .btn	*Smooth hover animation on all buttons*
L61-64	.btn-primary:hover { background:var(--accent); }	*Button turns accent red on hover*
L142	.cart-toggle:hover { background:var(--accent) }	*Cart button changes color on hover*
L201-203	.product-card:hover { box-shadow:var(--shadow-md) }	*Product card lifts on hover*
L438	transition: opacity .25s, transform .25s on .product-modal	*Modal slides in smoothly*
L473	transition: border-color var(--transition) on .modal-thumb	*Thumbnail border animates on select*

■ Why: Using --transition variable (L18) means all animations share the same speed. Changing it in :root instantly updates every hover effect site-wide.

FILE: script.js (775 lines total)

10. JavaScript: Arrays, Objects & Loops

Arrays hold lists. Objects hold structured data. Loops and array methods process them.

Lines	Code / Concept	Why Used
L8-54	const products = [{id, name, price, meta, images, reviews}, ...]	*Array of 6 product objects — product database*
L83	for (const product of products) { ... }	*for…of loop — iterates over all products*
L153	Object.entries(cart).map(([id, qty]) => ...)	*Convert cart object entries to array of products*
L160	.reduce((sum, entry) => sum + entry.lineTotal, 0)	*Array.reduce — sums cart total price*
L164	Object.values(cart).reduce((sum, qty) => sum + qty, 0)	*Counts total items across all cart entries*
L634	reviews.reduce((total, r) => total + r.rating, 0)	*Sums all ratings to calculate average*
L628	[...product.reviews, ...submitted]	*Spreads seed + submitted reviews into one array*

■ Why: Your entire app runs on one array (products L8-54). map() renders it (L191), filter() would search it, reduce() totals it (L160). Remove this array and nothing displays.

11. Variables & Functions

Functions make code reusable. const = immutable, let = reassignable (both block-scoped ES6).

Lines	Function / Variable	Why Used
L8	const products	*const — product data never reassigned*
L59	let cart = loadCart()	*let — cart state can be updated on every add*
L61-68	function loadCart() { return JSON.parse(...) }	*Reusable helper — called on page load*
L70-72	function saveCart() { localStorage.setItem(...) }	*Persists cart to browser storage*
L79-104	function renderProducts() { ... }	*Renders all product cards from data array*
L106-108	function formatPrice(amount) { return `■${...}` }	*Reusable price formatter — called in 6+ places*
L159-161	function getCartTotal() { return .reduce(...) }	*Calculates order total — called in cart & checkout*
L384-390	function getCurrentUser() { ... }	*Returns logged-in user object or null*

■ Why: renderProducts() (L79) is called once but loops through all 6 products. formatPrice() (L106) is called in 10+ places — without it you'd duplicate the ■ formatting logic.

12. ES6+ Features (Arrow, Template Literals, Spread, Destructuring)

Modern JavaScript syntax — cleaner, shorter, more powerful.

Lines	Feature / Code	Why Used
L113	(event) => { ... } Arrow function	*Shorter syntax; lexical 'this' binding*
L88-100	card.innerHTML = ` ... ${product.name} ...` template literal	*HTML strings with JS variables inline — no +*
L628	[...product.reviews, ...submitted]	*Spread — merges two arrays in one line*
L154-156	const { ...product, quantity, lineTotal } object spread	*Object spread — copies product + adds new fields*
L305-310	const order = { customer, items, total, placedAt } shorthand	*Object shorthand — key name = variable name*
L8	const products = [...]	*const — block scoped, safer than var*
L59	let cart = loadCart()	*let — block scoped, can be reassigned*
L677-679	product.images.map((src, index) => `...`)	*Arrow function inside Array.map() method*

■ Why: Template literals (L88-100) avoid messy string concatenation with + signs. Spread [...a, ...b] (L628) merges two arrays in one expression without .concat().

13. DOM Manipulation

DOM Manipulation = using JavaScript to read, create, and change HTML elements on a live page.

Lines	Code	Why Used
L77	document.getElementById('productGrid')	*Finds grid element by ID*
L81	productGrid.innerHTML = ""	*Clears grid before re-rendering*
L84	document.createElement('article')	*Creates a new product card DOM element*
L88-100	card.innerHTML = `...template literal...`	*Injects full HTML into each card*
L102	productGrid.appendChild(card)	*Adds the card to the live grid*
L180	cartCountEl.textContent = getCartCount()	*Updates cart badge number*
L185	cartItemsEl.innerHTML = `...`	*Replaces cart list with new items*
L671-673	modalProductName.textContent = product.name	*Fills in product name in modal*
L231-233	cartDrawer.classList.add('open')	*Adds CSS class to open cart drawer*
L697	.querySelectorAll('.modal-thumb').forEach(el => el.classList.remove('active'))	*Deselects all thumbnail borders*

■ Why: Without DOM Manipulation, pages are static. L84-102 is why product cards appear — the grid starts empty (productGrid.innerHTML='' at L81) and JS fills it dynamically.

14. Event Handling

Events connect user actions (click, type, submit) to JavaScript functions via addEventListener.

Lines	Event / Code	Why Used
L112-131	productGrid.addEventListener('click', (event) => {...})	*Delegate: one listener handles all card clicks*
L242-246	cartToggle.addEventListener('click', () => { ... })	*Toggle cart open/closed on Bag button click*
L247	cartClose.addEventListener('click', closeCart)	*Close cart drawer*
L248	cartOverlay.addEventListener('click', closeCart)	*Click backdrop to close cart*
L260	paymentSelect.addEventListener('change', updateWalletHint)	*Show wallet balance when wallet selected*
L273-335	checkoutForm.addEventListener('submit', (event) => { event.preventDefault()... })	*Checkout form submit + validation*
L463	loginForm.addEventListener('submit', (event) => { ... })	*Login form submit*
L749-766	reviewForm.addEventListener('submit', (event) => { ... })	*Review form submission*

■ Why: event.preventDefault() at L275 and L464 stops browser from reloading on form submit. Event delegation at L112 means one listener handles clicks on ALL 6 product cards.

15. Browser Storage (localStorage) & JSON

localStorage stores data in the browser permanently. JSON converts JS objects to strings.

Lines	Code	Why Used
L63-64	localStorage.getItem('stylehub-cart') / JSON.parse(...)	*Loads cart on page load; || {} is null fallback*
L71	localStorage.setItem('stylehub-cart', JSON.stringify(cart))	*Saves cart so it persists after refresh*
L352-357	localStorage.getItem('stylehub-users')	*Reads registered users object*
L365	localStorage.setItem('stylehub-users', JSON.stringify(users))	*Saves updated user records*
L369	localStorage.getItem('stylehub-current-user')	*Reads session — which user is logged in*
L373	localStorage.setItem('stylehub-current-user', username)	*Saves login session after successful login*
L377	localStorage.removeItem('stylehub-current-user')	*Clears session on logout*
L611	localStorage.getItem('stylehub-reviews') / JSON.parse	*Loads user-submitted reviews per product*

■ Why: localStorage replaces a real database here. JSON.stringify converts objects to strings for storage. JSON.parse reverses it. Without it, all data disappears on page refresh.

16. Authentication & User Accounts

Your project simulates login/signup using localStorage sessions.

Lines	Code / Logic	Why Used
L432-458	signupForm.addEventListener('submit', ...)	*Register: validates unique username, saves user*
L446-451	users[username] = { password, wallet:1000, address:{}, orders:[] }	*New user object with ■1000 default wallet*
L462-481	loginForm.addEventListener('submit', ...)	*Login: checks username+password in localStorage*
L470	if (!user || user.password !== password)	*Validates credentials — shows error if wrong*
L483-492	function performLogout() { clearCurrentUsername(); ... }	*Clears session, resets UI to logged-out state*
L530-551	function loadAccount() { ... }	*Shows auth panel vs account panel based on session*
L293-296	if (customer.payment === 'wallet') { if (!user) { ... } }	*Wallet payment blocked for non-logged-in users*

■ Why: This simulates real auth without a backend. The wallet feature (L293-302) requires login because wallet balance is stored per user in localStorage.

FILE: about.html

17. Semantic HTML in about.html

Lines	Tag / Code	Why Used
L1	!DOCTYPE html + html lang="en"	*HTML5 document with language for accessibility*
L11	body — shared structure with index.html	*Same header/footer reused across all pages*
~L20	<section class="about">	*Semantic: primary about content section*
~L22	<h2> heading inside section	*Proper heading hierarchy — h1 is in header*
~L25	<p> about/description text	*Plain paragraph — semantic prose content*

■ Why: section tells the browser this is a standalone thematic grouping. Proper heading hierarchy (h1 in header, h2 in sections) improves SEO ranking.

COMPLETE QUICK REFERENCE — All Topics & Lines

Topic	File	Lines
HTML5 Semantic Tags (header/nav/section/aside/footer)	index.html	*L14,L18,L37,L46,L113,L131*
Accessibility (aria-label / aria-expanded)	index.html	*L28,L58,L75,L113,L116*
Forms & Input	checkout.html	*L83-L107*
CSS Variables (:root)	style.css	*L7-19*
CSS Reset / Box Model base	style.css	*L24*
Box Model (padding/border/shadow)	style.css	*L44-55,L197,L210,L292-305*
Flexbox (header)	style.css	*L92-106*
Flexbox (product card body)	style.css	*L209-215*
CSS Grid (product grid)	style.css	*L189-193*
CSS Grid (checkout/account/modal)	style.css	*L306,L312,L380,L452*
Responsive / Media Queries	style.css	*L619-627*
Transitions & Hover States	style.css	*L53,L61-64,L201-203,L438*
Arrays & Objects (data)	script.js	*L8-54*
for…of loop	script.js	*L83*
Array.map() / .join()	script.js	*L153,L191,L677*
Array.reduce()	script.js	*L160,L164,L634*
Spread operator ([...])	script.js	*L628*
const / let variables	script.js	*L8,L59,L382*
Functions (helpers)	script.js	*L61-108*
Function (renderProducts)	script.js	*L79-104*
Function (formatPrice / getCartTotal)	script.js	*L106-161*
Arrow functions (=>)	script.js	*L113,L153,L677*
Template literals (backtick)	script.js	*L88-100,L191-209*
Object shorthand	script.js	*L305-310*
DOM: getElementById / createElement	script.js	*L77,L84*
DOM: innerHTML / textContent	script.js	*L81,L88,L180,L671*
DOM: classList manipulation	script.js	*L231,L237,L416,L697*
click / change events	script.js	*L112,L242,L260*
submit + e.preventDefault()	script.js	*L273-275,L463-464*
localStorage + JSON	script.js	*L63,L71,L352,L365,L373*
Authentication (login/signup forms)	script.js	*L432-481*
User wallet & orders	script.js	*L293-302,L446-451*

PE-1 Marks Breakdown

#	Component	Marks	Your Files
1	Presentation (Understanding + Clarity + Engagement)	5	*All files — be ready to explain any line*
2	HTML Structure & CSS Styling	8	*index.html + style.css + about.html*
3	JavaScript & GitHub Basics	8	*script.js + GitHub repo commits*
4	DOM Manipulation	5	*script.js L77-L131, L170-L219*
5	React Basics (JSX, Components, Props, State, Hooks)	4	*React mini-project (separate)*
TOTAL		30	

■ WARNING: GitHub repository MUST have regular commits. Evaluator WILL check commit history, code contributions and project progress. Missing or empty repo = mark deduction. Absent on evaluation day = ZERO marks for that component.

Good luck with your PE-1! Chitkara University | Dept. of CSE | B.E. 3rd Semester